

Actividades Semana 2 — Python para Ciencia de Datos

Guía del estudiante

Curso: Fundamentos para IA — Especialización en Inteligencia Artificial

En esta guía trabajamos con **Pandas** como librería principal. Numpy, SciPy, Matplotlib y Seaborn se ven en semanas posteriores — aquí el foco es carga, exploración, limpieza, agregación y combinación de datos.

Actividad 1 — Explorador de datos guiado

Dataset: Titanic (público, se carga directo por URL).

```
import pandas as pd

url =
"https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

print(df.shape)
print(df.columns.tolist())
print(df.head())
print(df.info())
print(df.describe())
```

Consigna:

1. ¿Cuántas filas y columnas tiene el dataset?
 2. ¿Qué columnas tienen valores nulos y cuántos?
 3. ¿Cuál es el promedio de edad de los pasajeros? ¿Y la mediana? ¿Por qué difieren?
 4. Sin usar gráficos (aún no toca), describe con `value_counts()` cómo se distribuyen los pasajeros por clase (`Pclass`) y por sexo (`Sex`).
-

Actividad 2 — Caza de errores

El siguiente código tiene **5 errores**. Encuéntralos y corrígelos sin reescribir el bloque desde cero.

```
df_titanic = pd.read.csv(url)
edad_promedio = df_titanic["Age".mean()]
menores = df_titanic[df_titanic.Age < 18]
menores["es_menor"] = True
sobrevivientes_por_clase = df_titanic.groupby("Pclass").sum("Survived")
print(df_titanic.sort_values(by="Fare", ascending=True[0:5]))
```

Entrega el código corregido y, para cada error, una línea explicando qué estaba mal y por qué.

Actividad 3 — Reto de limpieza de datos

Usando el mismo dataset de Titanic:

```
# 1. Identifica y cuantifica los nulos
nulos = df.isnull().sum()
print(nulos[nulos > 0])

# 2. Decide cómo imputar 'Age' y 'Embarked' – completa el código
df["Age"] = df["Age"].fillna(____)      # ¿media, mediana u otro criterio?
df["Embarked"] = df["Embarked"].fillna(____)

# 3. Decide si 'Cabin' se debe conservar o eliminar (tiene muchos nulos)
# tu código aquí

# 4. Verifica y elimina duplicados
print(df.duplicated().sum())
df = df.drop_duplicates()

# 5. Corrige los tipos de dato que consideres necesarios
```

Pregunta obligatoria de sustentación: justifica con la forma de la distribución (no con "porque así se hace normalmente") por qué elegiste esa estrategia de imputación para **Age**.

Actividad 4 — Groupby y agregaciones (caso de negocio)

```
import numpy as np
np.random.seed(7)
n = 500
productos = ['Laptop', 'Mouse', 'Teclado', 'Monitor', 'Audifonos']
regiones = ['Norte', 'Sur', 'Centro', 'Oriente']

ventas = pd.DataFrame({
    'id_venta': range(1, n+1),
    'producto': np.random.choice(productos, n),
    'region': np.random.choice(regiones, n),
    'unidades': np.random.randint(1, 10, n),
    'precio_unitario': np.random.choice([1200000, 45000, 90000, 650000, 60000],
n),
    'id_cliente': np.random.randint(1, 120, n)
})
ventas['total'] = ventas['unidades'] * ventas['precio_unitario']

# Ejemplo: producto más vendido (en $) por región
resumen = ventas.groupby(['region', 'producto'], as_index=False)['total'].sum()
top_por_region = resumen.sort_values('total',
```

```
ascending=False).groupby('region').head(1)
print(top_por_region)
```

Consigna: formula tú mismo otras dos preguntas de negocio (ejemplo: "¿cuál es el cliente con mayor gasto acumulado?", "¿qué región tiene el ticket promedio más alto?") y resuélvelas con `groupby` + `agg`.

Actividad 5 — Merge de múltiples fuentes

```
clientes = pd.DataFrame({
    'id_cliente': range(1, 121),
    'ciudad': np.random.choice(['Bogotá', 'Medellín', 'Cali', 'Cúcuta'], 120),
    'segmento': np.random.choice(['Retail', 'Corporativo'], 120)
})

fusion = ventas.merge(clientes, on='id_cliente', how='left')

# Antes de seguir: verifica que el merge no duplicó ni perdió filas
assert len(fusion) == len(ventas), "El merge duplicó o perdió filas – revisa las claves"

 analisis = fusion.groupby('segmento')['total'].agg(['sum', 'mean', 'count'])
print( analisis)
```

Actividad 6 — Mini-proyecto integrador

Con el dataset de ventas simulado (o uno propio si prefieres), entrega un script `.py` o notebook que:

1. Cargue y explore el dataset (shape, tipos, nulos).
2. Limpie: nulos, duplicados, tipos de dato correctos.
3. Responda **dos preguntas de negocio propias** usando `groupby/agg`.
4. Si necesita una segunda fuente, haga un `merge` con verificación de integridad (`assert` de conteo de filas).
5. Exporte el resultado final a CSV con `to_csv`.
6. Incluya un párrafo (no código) explicando qué decisión de limpieza tomaste y por qué.

Rúbrica de evaluación

Criterio	Insuficiente	Aceptable	Sobresaliente
Exploración inicial	No inspecciona antes de operar	Usa <code>.info()/describe()</code>	Además interpreta lo que encuentra
Limpieza de datos	Ignora nulos/duplicados	Los trata sin justificar	Justifica cada decisión con evidencia de los datos
Uso de <code>groupby/merge</code>	Sintaxis incorrecta o no aplica	Funciona pero sin verificación	Verifica integridad post-merge

Criterio	Insuficiente	Aceptable	Sobresaliente
Preguntas de negocio	Triviales o no formuladas	Formuladas y respondidas	Formuladas, respondidas y bien argumentadas
Código	No reproducible	Reproducible pero desordenado	Reproducible, comentado, con nombres claros

Actividad 7 (opcional) — Loop vs vectorización

```
import time

# Versión con loop explícito
t0 = time.time()
totales_loop = []
for i in range(len(ventas)):
    fila = ventas.iloc[i]
    totales_loop.append(fila['unidades'] * fila['precio_unitario'])
t_loop = time.time() - t0

# Versión vectorizada
t0 = time.time()
totales_vec = (ventas['unidades'] * ventas['precio_unitario']).values
t_vec = time.time() - t0

print(f"Loop: {t_loop:.4f}s | Vectorizado: {t_vec:.4f}s")
print(f"Vectorizado es ~{t_loop/t_vec:.0f}x más rápido")
```

Pregunta de cierre: ¿por qué la versión vectorizada es tan superior? No respondas "porque pandas es más rápido" — explica qué está pasando por debajo con la iteración en Python puro.

Quiz conceptual rápido (autoevaluación)

1. ¿Cuál es la diferencia entre `df[df.col > 5]` y `df.loc[df.col > 5]`?
2. ¿Por qué `df["col"]["Age"].mean()` produce un error distinto a `df["Age"].mean()`?
3. ¿Qué significa que un `merge` sea `how='left'` vs `how='inner'`, y cuándo cambiaría el número de filas resultante?
4. ¿Por qué imputar con la media es riesgoso si la distribución tiene outliers?
5. Menciona un caso donde `groupby().sum()` dé un resultado engañoso si no se filtran antes los nulos.

Datasets alternativos

- Ventas simulado (código de la Actividad 4)
- <https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv>
- Cualquier CSV propio de tu área de interés (opción válida para el mini-proyecto)